Project File Map

Русский · Reference · Install

Where everything lives. This document is for whoever maintains the project; a user does not need it.

Working folders (`input`, `output`, `logs`, `report`, `release`, `workspace`, `runtime`, `wheelhouse`) are described by role rather than file by file — their contents are produced by runs and rebuilt.

Root

File	What it is
`Start.exe`	Compiled window launcher: opens `App\AudionAddressProcessor.exe`
`address_cli.cmd`	The console command: the same operations without a window; the window itself stands on it
`builder_main.cmd`	Portable environment build
`cleanup_project.cmd`	Cleanup of generated artifacts
`pytest.ini`	Test settings
`release_manifest.json`	Manifest of the last release build
`README.md`	Public project description

CMD files must stay UTF-8 without BOM with CRLF line endings. After editing, run `install\Check-CmdEncoding.cmd`.

`App` and `Source` — The Window

Path	What it is
`Source/`	The window sources: Avalonia 12 / .NET 10. `Core/` — running the command, the manifest, field values, the journal, settings; `Views/` — the form, panels, dialogs; `MainWindow.*.cs` — sections, working area, running, OKTMO; `Ps/` — source of the program's own PowerShell; `Tools/` — icons and the check without a mouse
`Source/config/i18n.json`	The English pairs of every Russian text of the window
`App/`	The built window (`AudionAddressProcessor.exe`) and the own PowerShell `AudionPs.exe` for the build scripts; built by `install\Build-App.cmd`, travels with a release as it is

`config` — Settings

File	What it is
`project.yaml`	Engine settings: city, reference column, target columns, linked columns, safety limits
`window_layout.yaml`	The layout of the window: sections, the operations in them, the buttons of the working area and of the result panel, icons
`tool_manifest.yaml`	Every operation and field of the window; the interface is built from it
`version.json`	Version and the language of the root README
`path_history.json`	History and pins of source and target paths
`window_settings.json`	The window's settings: size, language, theme, font (made by the window itself)
`oktmo_current_keys.txt`	Current OKTMO search keys
`oktmo_region_pins.json`	Pinned regions
`oktmo_municipality_pins.json`	Pinned municipalities

File	What it is
`oktmo_pins_bundle.json`	The combined pin set

`system_core` — Code

Top Level

File	What it does
`Ultimate_GT_Aligner.py`	The matching engine: address parsing, passes, linked column transfer
`safe_table_join.py`	Comparison of two workbooks and column transfer through Excel
`excel_layout.py`	Result workbook formatting: column widths, text wrap, fill
`remove_empty_rows.py`	Removal of entirely empty rows
`Parser_Diagnostic.py`	Address parsing diagnostics
`inspect_md_tables.py`	Markdown table inspection
`doctor.py`	Environment and import checks
`main.py`	Direct matching run without the window
`cli.py`	The console command: `describe`, `run`, `options`, `oktmo`, `serve`, `stop-office`, `version`
`office_session.py`	Keeps track of an operation's own Excel and Word, so a stop closes only them

`address_engine` — The Address Engine

File	What it does
`audion_address_core.py`	Address decomposition, recognition of address-like text
`address_slots.py`	Slots: definition, filling, address line assembly
`slot_fill.py`	Filling slots from evidence
`models.py`	Data structures: observation, address parts, fingerprint, cluster

File	What it does
`source_intake.py`	Reading sources in every supported format
`source_table_assembly.py`	Collecting addresses from sources into a workbook
`source_cleanup.py`	Filtering out Office temporary files
`reference_builder.py`	Building the reference directory; slot and column definitions
`reference_processor.py`	Sorting and decomposing the directory by slots
`oktmo_lookup.py`	OKTMO index, disk cache, region / municipality / settlement lookup
`oktmo_user_keys.py`	The project key file and pins
`rosstat_update.py`	Downloading and updating the Rosstat registry
`region_reference.py`	Region reference
`russian_morphology.py`	Grammatical forms of names
`universal_address.py`	The common address normalizer
`normalizer_runtime.py`	Configuring the normalizer per run
`address_diagnostics.py`	Diagnostic extracts
`legacy_office_converter.py`	DOC/XLS conversion

`services` — The Service Layer

`address_aligner_service.py` is the only file. Every window command points to a function here: the service takes the parameters, calls the engine, and writes the log, the report, and the artifact list.

`core` — Core

File	What it does
`jobs.py`	Running operations, cancellation, progress, the live line of a long stage
`operation_model.py`	The rules that assemble the parameters of a run: defaults, conditions for showing fields, slot order; the manifest as the window takes it

File	What it does
`window_layout.py`	Reading `window_layout.yaml` and checking that every operation stands in the window
`manifest.py`	Parsing `tool_manifest.yaml`
`paths.py`	Managed project folders
`config.py`	Reading settings
`ansi_terminal.py`	Stripping the colour control codes from program output
`cmd_encoding.py`	CMD encoding checks
`output_decode.py`	Decoding subprocess output
`logging_utils.py`	Logging

Other

Path	What it is
`icons/`	The program icon
`start_launcher/`	Source of `Start.exe`

`install` — Install and Build

File	What it does
`Build_Portable_Env.cmd` / `.ps1`	Portable environment build
`install_portable_offline.cmd`	Install from the local `wheelhouse`
`verify_portable_env.cmd`	Verify the built environment
`init_folders.cmd`	Create the working folders
`Check-CmdEncoding.cmd`	Check CMD encodings
`Clean-Install-Cache.cmd` / `.ps1`	Clean the build cache

File	What it does
`Update-Requirements-Lock.cmd`	Update the dependency lock file
`Build-StartLauncher.cmd` / `.ps1`	Build `Start.exe`
`Build-App.cmd`	Build the window `App\` from `Source\`
`requirements_full.in`	Dependency list
`README_INSTALL.md`	Install notes

`data` — Data

Path	What it is
`regions_reference.yaml`	Region reference
`rosstat/data-*.csv`	The OKTMO registry, downloaded by a button in the window
`rosstat/oktmo_lookup_cache.json`	The parsed OKTMO index

`docs` — Documentation

File	What it is
`README_RU.md`, `README_EN.md`	Project description, the entry point
`USER_GUIDE_RU.md`, `USER_GUIDE_EN.md`	User guide
`OKTMO_RU.md`, `OKTMO_EN.md`	The OKTMO registry, keys, and pins
`REFERENCE_RU.md`, `REFERENCE_EN.md`	Technical reference
`INSTALL_RU.md`, `INSTALL_EN.md`	Install and launch
`tools/PROJECT_FILES_GUIDE_*.md`	This document

File	What it is
`tools/WINDOW_INVENTORY.md`	The inventory of the earlier window (NiceGUI) and its check against the new one: what is covered where (in Russian)
`PDF/`	Rendered PDFs of every document in two themes

PDFs are built by a separate tool from every `.md` in `docs`, nested folders included. The folder structure is preserved.

`tests` — Tests

File	What it covers
`test_address_aligner_regression.py`	The matching passes
`test_safe_table_join.py`	Moving columns by address
`test_excel_layout.py`	Result workbook formatting
`test_console_output.py`	Stripping colour codes from output, the encodings of subprocess output
`test_cli_door.py`	The console command: the same run parameters as the earlier window, stopping, Excel and Word
`test_window_avalonia.py`	The window and the command read the same keys; the English pairs; the self-check of the built window
`former_window_parameters.json`	The parameters of the 14 operations recorded from the earlier window on 11.10.2026: the command is checked against them

The window is checked not by a page in a browser but by itself: `App\AudionAddressProcessor.exe --self-check` builds every section and compares them with the manifest.

`licenses` — Licenses

License texts of third-party components, `THIRD_PARTY_NOTICES.txt`, `COMPONENTS.json`, the scan report, and the pack manifest. Produced by the Audion Build Licenses tool from the finished contents

of the project, not edited by hand.